A PRACTICAL BOOKLET

VIBE CODING

using Cursor AI

Describe the behavior. Let the agent draft it. Run the result, read the diff, and steer with evidence. The typing can move. The responsibility does not.

01 Name the outcome

02 Point at the real code

03 Let Agent draft

04 Read the diff

05 Run it

06 Steer with the failure

INDEPENDENT

Not a Cursor manual. Shortcuts checked against public docs, October 2026.

Seventeen pages. The desk card is on 16.

Field guide · October 2026

HOW TO USE THIS

Read it once. Keep page 16.

This is a field guide for people who already write software and want Cursor's agent to take the first draft. It is not a tour of every setting.

03 **What it is**
The sentence that named the style, and the three jobs you still do.

04 **The loop**
Six moves, and the only kind of follow-up that helps.

05 **Three speeds**
Tab, Agent, and Cloud Agents. When each one is the right handoff.

06 **Four modes**
Agent, Ask, Plan, Debug. Shift+Tab, and when to start a new chat.

07 **Prompts that finish**
Outcome, location, constraint, proof.

08 **Context**
@ mentions, screenshots, and the ring that shows the window filling up.

09 **Rules and skills**
What belongs in every chat, and what you invoke with /.

10 **One session**
A full pass on a small app: plan, then build.

11 **Steering and rollback**
Queue, interrupt, checkpoints, side chats.

12 **Cloud Agents**
Same loop, another machine, a branch you review later.

13 **You still own the diff**
Five checks before you keep the work.

14 **When it goes wrong**
Eight familiar failures, and the move that stops each one.

15 **Prompt pack**
Eight prompts you can paste and adapt.

16 **Desk card**
Keys, modes, and the if/then.

17 **The next hour**
One exercise on a repo you already know. Then the sources.

DO THIS

Work a real change while you read pages 10 and 11. The loop only sticks if you steer once.

SKIP THIS

Pricing, admin consoles, and the full catalog of tools. Those move, and they are not the practice.

IF A KEY DOES NOTHING

Trust the current screen and cursor.com/docs. The names here match the docs as of October 2026.

WHAT IT IS

Name the behavior. Keep the blame.

Vibe coding means you describe the outcome and let a model draft the code. You stay in the loop by running the result and correcting it, the way you would stop a take and point at the wrong note.

> *“There’s a new kind of coding I call ‘vibe coding’, where you fully give in to the vibes, embrace exponentials, and forget that the code even exists.”*
>
> Andrej Karpathy · February 2025

That sentence named the style. Karpathy was talking about agents in the editor, Cursor among them, and about going so fast he stopped reading. The Saturday version of that is real: a prototype, a spike, a script you will throw away. A repository other people ship is a different job. This booklet keeps the name and drops the part about not reading.

Saturday

Accept the draft, run it, throw it away if the feeling is wrong. Fine for a toy, a one-off script, or a sketch nobody else will maintain.

A repo you ship

The agent still drafts. You decide the boundary, you read the diff, and you can explain the change without the chat open. That is the practice in the rest of these pages.

DECIDE

What should change, and what must not. If you cannot say it, Plan mode is the right place. Agent mode will guess.

CONSTRAIN

Which files, which existing pattern, which command proves it. Constraints are how a vibe becomes a task.

VERIFY

The diff, the test or the screen, and one sentence you could say out loud. A green compile is not that sentence.

SIX MOVES

Steer shorter than you started.

01 Name the outcome

One sentence a stranger could test. “Show this month’s total and ignore transfers,” not “improve the dashboard.”

02 Point, or let it search

@ a file when you know it. If you do not, say so. A wrong file is worse than no file. Agent can search the repo.

03 Let it draft

Agent edits while you watch. There is no fixed cap on tool calls. Stop the run if it walks into the wrong directory.

04 Read before you admire

Look at the diff as it lands. Reject files that are not the task. A tidy summary is not a review.

05 Run the thing

Tests, the command you named, or the screen. Agent can drive the browser and check a page. Ask it to.

06 Steer with evidence

The next message should be shorter. Paste the failure. Do not restate the essay. The chat already has it.

A SECOND MESSAGE THAT WANDERS

“That’s not quite right. Can you make it cleaner and also think about edge cases and maybe the design?”

A SECOND MESSAGE THAT STEERS

“Total is 2600. fixtures/march.ts has two Transfer rows. Expected 1840. Don’t edit the fixture.”

WHERE THE HANDS GO

You are not always handing over the wheel.

Cursor gives you three speeds. Mixing them up is how a two-line edit becomes a tour of the repo.

01 Tab

You already know the shape. You want the keystrokes to disappear.

Tab is autocomplete. It watches recent edits, the code around the cursor, and linter errors, then suggests the next change in gray. Tab accepts it. Esc rejects it. Ctrl+Right on Windows and Linux, or Cmd+Right on Mac, takes the suggestion a word at a time. Press Tab again and it jumps to the place it thinks you should edit next, including another file, shown in a portal at the bottom of the editor. Snooze it from the Tab indicator at the bottom right when you need to type uninterrupted. Multi-line edits and a missing import are normal, not a special mode.

02 Agent

The change spans files, or you would otherwise spend the time hunting.

Open the panel with Ctrl+I on Windows and Linux, or Cmd+I on Mac. Speak in plain language. Agent searches, edits, runs the terminal, and can open a browser to check what it changed. Select code and press Ctrl+L or Ctrl+K (Cmd+L or Cmd+K on Mac) to open Agent with those lines attached. Inline edit used to live on Cmd+K. That path now sends the selection to Agent instead. Watch the diff. Click Stop if the direction is wrong. Review in the diff and reject hunks you do not want.

03 Cloud Agent

The task is described, and you want a branch more than a live pair.

A Cloud Agent runs the same kind of work on its own machine. It clones the repo, uses a development environment, and can open a pull request. You can run many at once, from the editor or from cursor.com/agents, and your laptop does not have to stay online. It can only finish what that machine can run. Page 12 is the decision.

AGENT, ASK, PLAN, DEBUG

Pick the mode before you pick the words.

Press Shift+Tab in the Agent panel to cycle modes, or use the mode picker. Each mode has its own context, so a switch starts a fresh window. Rules still apply in all four.

Edits files

Agent

The default. Features, refactors, bug fixes, tests, shell commands. Use it when you know the task well enough to recognize a wrong file.

Most of a working day belongs here. Do not drop into it for a system you have not even located.

Read only

Ask

Explanations and maps. It does not edit. Use it to trace a path before anyone touches it.

“Follow the export button to the file on disk. Cite the functions. Don’t change anything.”

Plan, then you approve

Plan

For a change with more than one reasonable shape, or one that crosses several files. Agent researches, asks, and writes a plan you can edit. Nothing is built until you say so. Cursor will also suggest Plan when the prompt sounds large.

Plans save outside the repo by default. Use Save to workspace if the plan should live with the code. If the build drifts, revert and tighten the plan. That beats four corrective messages.

Edits, after evidence

Debug

For a bug you cannot settle by reading. Use it when the failure shows up only at runtime and you need logs, a reproduction, or both, before a fix.

Bring the symptom and what you already tried. A Debug thread is still a place to say what must not change.

One task, one chat.

A thread about a tax calculation should not also redesign the navigation. When the job changes, start a new chat and carry the decision across in a few lines: what is already true, what is still wrong, what not to touch, and how you will know. Switching modes will not clean a muddy thread. It starts fresh context, and it drops the trail you actually needed.

A PROMPT THAT CAN FINISH

Four lines, even inside a paragraph.

OUTCOME	What a stranger would observe. A number, a screen, a passing command. Not a mood.
LOCATION	A path, or “search, and do not invent a new module.” Naming the wrong file is a location too. Leave it off if you are unsure.
CONSTRAINT	What stays: a fixture, a public API, a layout, a file you are not willing to restyle.
PROOF	The test command, the click, or the value you expect to see. The agent should run it, not describe it.

HARD TO FINISH

“Make the export better and fix the PDF. It feels off.”

POSSIBLE TO FINISH

“In PdfExporter, page-two images are clipped. Keep Urdu font embedding. Add a check that a two-page document still has content on both pages. Don’t restyle the layout XML.”

Specific beats long. If you have six constraints, write a list. If you are still choosing the design, you are not ready for a build prompt. Switch to Plan and stop there.

Paste a screenshot when the bug is visual. Drag it into the chat, or paste with Ctrl+V or Cmd+V. Say what is wrong and what the screen should do. The microphone in the input will take a spoken prompt. Read the transcription before you send it. Spoken file names are easy to get wrong.

You can change models mid-chat from the model picker. A faster model is enough for a tight edit. A more capable one is worth it when the change crosses files or the plan itself is the hard part. The docs describe Cmd+/ on Mac as a way to cycle models.

WHAT THE MODEL CAN SEE

The repo is not in its memory.

Every chat has a fixed window. Cursor draws it as a ring next to the input. Click the ring and you get a split: system prompt, tools, rules, skills, MCP, subagents, the summary of older turns, and the live conversation.

When the window fills, older turns are compressed into a summary. That summary is a lossy copy. Do not keep a precious constraint only in message three of a long thread. Put standing constraints in a rule. Put the current proof in the latest message.

ATTACH THIS	WHEN
`@file` or `@folder`	You know where the change lives. Type @ and keep typing. After a folder, type / to go deeper.
`@Terminals`	The interesting part is output already on screen. Attach it instead of describing it.
`@Commit` · `@Branch`	The question is about uncommitted work, or the whole branch against main.
`@Browser`	The proof is the running page. Agent can also drive the browser itself.
`@Chats`	A previous conversation has a decision you do not want to retype.
Nothing	You do not know the file. Say “search.” Agent can find files by name and by text. A confident wrong @ is an instruction to ignore the real one.

RULES

Included on every turn they apply to. A vague always-on rule taxes every message. Keep those few and short. Page 09.

SKILLS

Loaded when they match, or when you type /. Right place for a procedure you do not want in every chat.

MCP

External tools and data. Useful, and they occupy the same window. Connect what this repo actually needs.

Paste a stack trace whole. Do not summarize an error you have in front of you. Images of a broken layout belong in the prompt, not in a prose recreation of the pixels.

STANDING INSTRUCTIONS

Say it once. Stop repeating it.

Models do not keep a memory between chats. Rules and skills are how a project remembers. User rules follow you across repos. Team rules, on Team and Enterprise plans, follow the team. Project rules follow the repository, which is what you want for anything a teammate or a Cloud Agent should obey.

Project rules are `.mdc` files in `.cursor/rules`. A plain `.md` file in that folder is ignored. If you want plain markdown, use `AGENTS.md`.

ALWAYSAPPLY	DESCRIPTION	GLOBS	INCLUDED
`true`	–	–	Every chat. Globs are ignored.
`false`	–	set	When a matching file is in context.
`false`	set	–	When the agent judges it relevant.
`false`	–	–	Only if you @ mention the rule.

A RULE THAT EARNS ITS PLACE

```
---
description: PDF export stays paginated and bilingual
globs: app/src/main/java/**/PdfExporter.kt
alwaysApply: false
---
When changing PDF export, keep Urdu font embedding.
Paginate. Do not scale a page to fit.
If you touch this file, add or update a check that a
two-page document still has content on both pages.
```

A skill is a folder with a `SKILL.md`. Project skills live in `.cursor/skills/` or `.agents/skills/`. The description is how Agent decides to use it. Type `/` to run one for a single message. In the Agents Window and the CLI, Option+Enter on Mac or Alt+Enter on Windows and Linux keeps that skill on as a mode for the whole session.

While you are building the habit, four built-ins matter: `/create-rule`, `/create-skill`, `/review`, and `/review-bugbot`. `/review-security` is the pass aimed at vulnerabilities. They are a second reader. Page 13 says what they do not replace.

LEDGER, A SMALL EXPENSE LIST

Plan it, then let it build.

The home screen should show this month’s total above the transactions. Rows whose category is Transfer do not count. You have not decided where the number is computed. That is a Plan, not a build.

1 · PLAN MODE · SHIFT+TAB UNTIL IT SAYS PLAN

```
Plan only. Do not edit yet.

On the Ledger home screen, show this month’s total
above the list. Exclude transactions whose category
is Transfer. Use the existing transaction type.
Do not add a new page or a new state library.

Name the files you will change, where the total is
computed, and how I can verify the number in the
browser.
```

Read the plan for three things: it names files that exist, the total has a home in current code, and it did not wander into a redesign. Edit the plan in place if the file list is wrong. Save it to the workspace if you want the decision in the repo.

2 · BACK IN AGENT · ONLY AFTER THE PLAN IS RIGHT

```
Build the plan we just agreed.

Add a test beside the other transaction tests.
The March fixture includes Transfer rows.
The total must exclude them.

Run the test script. If a command fails, stop and
quote the error. Do not edit the fixture to make
the number match.
```

Then watch the diff, not the narration. The next page is what you do when the number comes back wrong. That is the normal case, not a failure of the method.

Example continues on page 11.

THE TEST FAILED, WHICH IS USEFUL

Quote the number. Freeze the fixture.

3 · SAME CHAT · THE FAILURE IS THE PROMPT

```
Failure:

Expected 1840, got 2600.
src/fixtures/march.ts has two Transfer rows,
400 and 360. The total is still summing them.

Exclude category === "Transfer" in the function
the plan named. Do not edit the fixture.
Do not reformat unrelated files.
Run the same test again.
```

This message works because it has the evidence, the invariant, and the same proof. It does not renegotiate the feature.

While it runs

In the editor, Enter queues your next note until the turn finishes. Ctrl+Enter, or Cmd+Enter on Mac, sends it now. Stop ends a turn that has gone into the wrong directory. On cursor.com/agents, and where the Agents Window has the newer control, Send now or a second Enter steers at the next tool call instead of cutting a command in half. Tab queues there. In the CLI, Enter steers at a safe boundary, and Enter again interrupts.

If you need a tangent

Type /side or /btw so a side question does not knock the total out of the main thread. A side chat keeps its own transcript and can see the parent. Pull it back later with an @ mention if the answer mattered. A long-lived aim can be a /goal, for example “get the transaction tests green and leave the fixture unchanged.” As of October 2026, /goal was still rolling out. If it is missing, write the same sentence as a normal prompt and do not change the subject.

Checkpoints roll files back. Git keeps what you meant.

Agent snapshots files before large edits. Restore Checkpoint, on an earlier message, returns the files to that point and leaves the conversation where it is. Checkpoints stay on your machine. They are not commits. If the turn went wide, restore, then rewrite the prompt. Do not spend four messages sanding a diff that started from the wrong plan. Commit when the sentence on page 13 is true.

LEAVING THE LAPTOP

The environment is the product.

A Cloud Agent is the same loop on a separate machine. It clones the repository, works on a branch, and can open a pull request for you to review. Start one from the editor or at cursor.com/agents. Several can run at once. The laptop can sleep.

It can use the browser, and it can use MCP servers that are available to it. It can also work across more than one repo when a task actually spans them. None of that helps if the machine cannot install dependencies, run the tests, or reach the service the code calls. Set the environment up before you expect finished work. The configuration can live in `.cursor/environment.json`, and a saved build is what later agents start from.

Send it to the cloud

The task is a feature or a fix you can describe completely. You want a branch and a pull request, not a partner looking over your shoulder. You are willing to review later, the way you would review anyone else's pull request.

Keep it local

You are still choosing the approach. You need to poke the interface every minute. The change is a few lines in a file you already have open. Tab or a local Agent is the faster loop.

For a body of work large enough to split, a Project lets a coordinating agent plan the job and hand pieces to other agents. Use that when you would otherwise write one enormous prompt. Do not use it to avoid deciding what the pieces are.

Rules in the repository travel with the repository, so a Cloud Agent sees them. Personal skills under your home directory stay on your machine unless you turn on sync for Cloud Agents you start yourself. Synced personal skills are still private to you. Anything a teammate's agent should follow belongs in the repo: `.cursor/rules`, `AGENTS.md`, or `.cursor/skills`.

> *An agent that can edit but cannot run the software has not closed the loop. It has typed.*
>
> Write the prompt as if you will not be at the keyboard to paste the error.

YOU STILL OWN THE DIFF

Five checks before you keep it.

Agent applies edits as it goes. Your review happens in the diff, not in the closing paragraph. Reject a hunk you did not ask for. For a pull request, a second pass is worth running. It is not a substitute for the list below.

01 **The behavior matches the sentence**

The thing you named is the thing that changed. Adjacent polish does not count as the task.

02 **Surprise files are justified or gone**

A drive-by rename, a format of an untouched directory, a new helper “for later” — send it back. Restore the checkpoint if the turn went wide.

03 **Tests were not softened**

A deleted assertion, a widened tolerance, or a fixture edited so the bug becomes the spec. If the test moved, read that hunk first.

04 **Nothing secret got copied in**

No tokens, no pasted production URLs with credentials, no logs of real user data added “just for debugging.”

05 **You can say it in two sentences**

Close the chat. Explain the change out loud. If you need the transcript to do it, you are not done. Switch to Ask and have it explain the diff. Then decide.

A second reader, not an owner.

Type /review and Cursor picks a review agent. /review-bugbot runs Bugbot against likely bugs and regressions. /review-security looks for vulnerabilities. Bugbot can also comment on pull requests. Use them after the five checks, especially on a branch you did not watch being written. They will miss things. They will also catch the sort of mistake a tired read skips. Neither one makes sentence five optional.

WHEN IT GOES WRONG

Recognize the shape. Change the prompt, not the volume.

Turning the same instruction up louder rarely helps. Match the failure, then make one change.

WHAT YOU SEE	WHAT HAPPENED	THE MOVE
A huge diff and a confident summary	The prompt had a mood and no boundary.	Restore the checkpoint. Add the file list and the proof. Build again.
The test changed so the run is green	The goal you implied was “passing,” not “correct.”	Say the fixture and the expected value are frozen. Re-run the original command.
It calls a helper that does not exist	It guessed an API from the shape of the name.	Paste the real signature, or switch to Ask and locate it before another edit.
Edits keep landing in the wrong package	The chat is stale, or two things share a name.	New chat. @ the directory. State what the other package is not for.
The same error, three times	It cannot see the failure you can see.	Paste the full output. Tell it to try once, then stop and report. No more blind edits.
It compiles. You still feel lost.	You skipped the read because the summary sounded finished.	Ask mode: “Explain this diff in ten lines. Don’t edit.” Then keep or restore.
A small fix rewrote the formatting of the file	You never said the untouched lines were out of scope.	Reject the noise. Add a short rule, or say it in the next prompt. /create-rule will draft one.
Plan mode built the wrong product, faithfully	The plan was vague and you approved it.	Revert. Edit the plan until the file list is something you would defend. Build once.

Agent can also split work out to subagents — research, shell, browser — each with its own window, reporting back to the chat. You can add your own under `.cursor/agents/`. That is useful when one thread is doing too many jobs. It is not a fix for a prompt that never said what done means.

PASTE, THEN REPLACE THE BRACKETS

Eight prompts.

1 Locate, in Ask mode

```
Trace how [symptom] happens, from the entry point
to the line that decides it. Cite files and functions.
Do not edit.
```

2 Plan, and stop

```
Plan only. Do not edit.
[One sentence outcome.]
Leave alone: [files, APIs, or layout].
Name the files you will change and the command
or click that proves it.
```

3 Build what you already agreed

```
Implement the plan we agreed.
Run [command]. If it fails, stop and quote the
error. Do not change tests to hide a failure.
```

4 A screenshot is the bug report

```
The attached screen is wrong. It shows [what].
It should [what instead]. Match the existing
components. Do not restyle the page.
```

5 Freeze the fixture

```
[paste the failure]
The fixture and the expected value stay as they are.
Fix production code only. Run [command] again.
```

6 Hand a fresh chat the state

```
Goal: [...]
Already true: [...]
Still wrong: [...]
Do not touch: [...]
Proof: [command or click]
```

7 Review the branch you are about to keep

```
/review
Review this branch against the base. Rank findings
by user-visible severity. Call out weakened tests
and files that look unrelated to the task.
```

8 Turn the last mistake into a rule

```
/create-rule
We had to undo a change that reformatted files
outside the task. Add a short project rule: only
format files already being edited, and say so in
the summary. Do not make it always-on unless it
must apply to every chat.
```

TEAR THIS PAGE OUT, MENTALLY

Keys, modes, and the if.

KEYS · WINDOWS/LINUX · MAC

Open Agent	Ctrl+I · Cmd+I
Cycle modes	Shift+Tab
Selection into Agent	Ctrl+L / Ctrl+K · Cmd+L / Cmd+K
Accept a Tab suggestion	Tab
Reject a Tab suggestion	Esc
Accept one word of Tab	Ctrl+Right · Cmd+Right
Jump to the next edit	Tab again
Queue a follow-up	Enter, while it runs
Send the follow-up now	Ctrl+Enter · Cmd+Enter
Stop the turn	Stop button
Side chat	/side · /btw
Run a skill	/
Keep a skill on as a mode	Alt+Enter · Option+Enter
Find in this chat	Ctrl+F · Cmd+F
Cycle models (Mac docs)	Cmd+/

On cursor.com/agents, Enter twice steers at the next tool call and Tab queues. In the editor, Enter queues and Ctrl+Enter or Cmd+Enter sends now. Page 11 has the longer version. Restore Checkpoint rolls files back and leaves the chat. Git is what you keep on purpose.

IF THIS IS TRUE

You can see the line	Tab
You can see the task	Agent
Several designs would fit	Plan
You cannot find the code	Ask
It fails only at runtime	Debug
You want a pull request	Cloud
The chat changed jobs	New chat
The diff went wide	Checkpoint

A PROMPT

Outcome.
Location, or “search.”
What must not change.
The proof you will run.

A FOLLOW-UP

The evidence, pasted.
What stays frozen.
The same proof again.
No second feature in the same breath.

TO KEEP IT

Behavior matches the ask. No surprise files. Tests not weakened. No secrets. You can explain it with the chat closed.

SECOND READER

/review
/review-bugbot
/review-security
After the five checks. Not instead of them.

DO THIS ON A REPO YOU KNOW

One hour. One bug you can already see.

1. Write three short project rules: how to run the tests, where new code should go, and what not to restyle. Keep each under eight lines. Use /create-rule if you want a draft, then cut it.
2. Pick a bug you can reproduce without the agent. If you cannot show the failure, you cannot steer.
3. Ask mode. “Trace this from the entry point to the line that decides it. Don’t edit.”
4. Plan mode. “Plan the smallest fix. List the files. Do not edit yet.” Argue with the plan until the file list looks right.
5. Build it. Name the command. If it fails, paste only the failure and what must stay frozen.
6. Read the diff with the five checks on page 13. Delete anything that is not the fix. Commit when you can explain it in two sentences.

SOURCES AND LIMITS

Product behavior in this booklet was checked in October 2026 against Cursor’s public documentation: Agent, Tab, Plan mode, prompting, rules, skills, and Cloud Agents, at cursor.com/docs. Shortcuts, mode names, and where a control lives all move. When this page and the product disagree, the product is right.

Andrej Karpathy named “vibe coding” in February 2025. The sentence on page 03 is his. The practice in the rest of the booklet — read the diff, freeze the fixture, keep the blame — is the part worth keeping after the prototype.

This is an independent field guide. It is not published by Cursor or Anysphere, and it is not affiliated with Karpathy. It leaves out pricing, enterprise administration, and the full tool catalog on purpose.

Rebuild the PDF from `docs/vibe-coding/source.html` with `docs/vibe-coding/build.sh`.